where $\circ$ denotes pointwise multiplication.

Define

$$D_i = P_{i:}^T dP_{i:} = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j = do_i^T o_i, \tag{4}$$

then

$$dS_{i:} = P_{i:} \circ dP_{i:} - D_i P_{i:}.$$

Hence

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij}(dP_{ij} - D_i).$$

Now we can get the gradients $\mathbf{dQ}$ and $\mathbf{dK}$. Recall that $S_{ij} = q_i^T k_j$, so

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij}(dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) k_j. \tag{5}$$

Similarly,

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij}(dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) q_i. \tag{6}$$

Therefore the backward pass can also be computed with $O(n)$ extra memory:

1. Compute $dv_j$ for all $j$ according to Eq. (3), which takes $O(d)$ extra memory.
2. Compute $D_i$ for all $i$ according to Eq. (4), which takes $O(n)$ extra memory.
3. Compute $dq_i$ for all $i$ according to Eq. (5), which takes $O(d)$ extra memory.
4. Compute $dk_j$ for all $j$ according to Eq. (6), which takes $O(d)$ extra memory.

### B.3 FlashAttention: Forward Pass

We describe the full details of FlashAttention forward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \tau \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{S}^{\text{masked}} = \text{MASK}(S) \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}) \in \mathbb{R}^{N \times N},$$
$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}} \mathbf{V} \in \mathbb{R}^{N \times d},$$

where $\tau \in \mathbb{R}$ is some softmax scaling (typically $\frac{1}{\sqrt{d}}$), MASK is some masking function that sets some entries of the input to $-\infty$ and keep other entries the same (e.g., key padding mask when sequences in the batch don't have the same lengths and are padded), and $\text{dropout}(x, p)$ applies dropout to $x$ elementwise (i.e., output $\frac{x}{1-p}$ with probability $1-p$ and output 0 with probability $p$ for each element $x$).

The full algorithm is in Algorithm 2. We save the output $\mathbf{O}$, the softmax statistics $\ell$ and $m$, and the pseudo-random number generator state $\mathcal{R}$ for the backward pass.

**Algorithm 2** FlashAttention Forward Pass

**Require:** Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size $M$, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability $p_{\text{drop}}$.

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into $T_r$ blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\ell$ into $T_r$ blocks $\ell_i, \dots, \ell_{T_r}$ of size $B_r$ each, divide $m$ into $T_r$ blocks $m_1, \dots, m_{T_r}$ of size $B_r$ each.
**for** $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  **for** $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
    On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
    Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
  **end for**
**end for**
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

## B.4 FlashAttention: Backward Pass

We describe the full details of FlashAttention backward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, the output $\mathbf{O} \in \mathbb{R}^{N \times d}$, and the output gradient $\mathbf{dO}$, we want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$.

We first describe the standard attention backward pass in Algorithm 3 for completeness.

**Algorithm 3** Standard Attention Backward Pass

**Require:** Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dS}\mathbf{K}$, write $\mathbf{dQ}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We now make two observations about FlashAttention backward pass:

1. We do not need to store the dropout mask of size $O(N^2)$ from the forward pass. Instead, we can save the pseudo-random number generator states from the forward pass and re-generate the dropout mask in the backward pass. This allows us to only use $O(N)$ extra memory.

2. When computing the softmax gradient, we use Eq. (4) to compute $D_i = P_{i:}^\top dP_{i:}$ without reducing over $P_{i:}$ and $dP_{i:}$ of size $N$ (they might not fit into SRAM). Instead we can rewrite $D_i = do_i^\top o_i$ and compute the dot product between vectors of size $d$.

The full FlashAttention backward pass algorithm is in Algorithm 4. Conceptually it is just a block version of the derivation in Appendix B.2.

**Algorithm 4** FlashAttention Backward Pass

**Require:** Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size $M$, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability $p_{\text{drop}}$, pseudo-random number generator state $\mathcal{R}$ from the forward pass.

Set the pseudo-random number generator state to $\mathcal{R}$.

Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

Divide $\mathbf{O}$ into $T_r$ blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into $T_r$ blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide $\ell$ into $T_r$ blocks $\ell_i, \dots, \ell_{T_r}$ of size $B_r$ each, divide $m$ into $T_r$ blocks $m_1, \dots, m_{T_r}$ of size $B_r$ each.

Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into $T_r$ blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to $T_c$ blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.

**for** $1 \le j \le T_c$ **do**

  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.

  Initialize $\tilde{\mathbf{dK}}_j = (0)_{B_c \times d}, \tilde{\mathbf{dV}}_j = (0)_{B_c \times d}$ on SRAM.

  **for** $1 \le i \le T_r$ **do**

    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.

    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.

    On chip, compute $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability $p_{\text{drop}}$.

    On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).

    On chip, compute $\tilde{\mathbf{dV}}_j \leftarrow \tilde{\mathbf{dV}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.

    On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).

    On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.

    On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.

    Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.

    On chip, compute $\tilde{\mathbf{dK}}_j \leftarrow \tilde{\mathbf{dK}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.

  **end for**

  Write $\mathbf{dK}_j \leftarrow \tilde{\mathbf{dK}}_j, \mathbf{dV}_j \leftarrow \tilde{\mathbf{dV}}_j$ to HBM.

**end for**

Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We see that similar to the forward pass, the backward pass performs $O(N^2)$ FLOPs and only requires $O(N)$ extra memory beyond inputs, output, output gradient, and input gradients.

We analyze the IO-complexity of the backward pass, similar to the forward pass (Theorem 2).

**Theorem 5.** *Let $N$ be the sequence length, $d$ be the head dimension, and $M$ be size of SRAM with $d \le M \le Nd$. Standard attention (Algorithm 0) backward pass requires $\Theta(Nd + N^2)$ HBM accesses, while* FlashAttention *backward pass (Algorithm 4) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.*

The proof is in Appendix C.